דוט נט תשפ"ו - פרויקט - שלב 4

דוט נט תשפ"ו - פרויקט - שלב 4 1
מטרות השלב 2
הנחיות לביצוע השלב והגשתו 2
ראשי פרקים - שלב 4 2
הנחיות פרטניות – שלב 4 - עדכון DAL לעבודה עם תבניות עיצוב 3
פרק 1 - הפיכת המחלקות הקיימות DalXml, DalList ל Singleton 3
פרק 2 - הגדרת מחלקת יצרן (Simple Factory) עבור שכבת הנתונים 3
פרק 3 - בדיקה חוזרת של שכבת הנתונים המעודכנת בעזרת הפרויקט הקיים DalTest 7
הנחיות פרטניות - שלב 4 - בניית שכבת הנתונים BL 8
הקדמה - שכבת הנתונים BL 8
פרק 4 - יצירת השכבה הלוגית - הוספת 2 פרויקטים BL ו- BlTest 8
פרק 5 - יצירת 4 תתי ספריות תחת פרויקט BL 9
פרק 6 - הגדרת ישויות הנתונים הלוגיות (חוזי נתונים - מחלקות) - BO 10
1. עמדו עם עכבר ימני על הספריה BO והוסיפו מחלקה ציבורית בשם Config 11
פרק 7 - הגדרה ומימוש של מחלקות עזר סטטיות לשימוש פנימי בשכבה הלוגית 12
פרק 8 - הגדרת ישויות השירות הלוגיות (חוזי השירות - ממשקים) - BlApi 17
פרק 9 - מימוש ממשקי השכבה הלוגית (מימוש חוזי השירות הלוגיים) 19
פרק 10 - הגדרת מחלקת יצרן (Simple Factory) עבור השכבה הלוגית 20
פרק 11 - הגדרת חריגות עבור השכבה הלוגית בתוך פרויקט BL 21
פרק 12 - בניית תוכנית בדיקה BlTest 22
פרק 13 - תמונת מצב 24
פרק 14 - יצירת תג והגשת שלב 4 במודל 25
נספחים - שלב 4 25
נספח 1 - דוגמאות לישויות נתונים לוגיות 25
נספח 2 - דוגמאות להגדרה ומימוש של מתודות עזר לוגיות 26
נספח 3 - דוגמאות למימוש מתודות של ממשק לוגי ע"פ הכללים 27

מטרות השלב

בשלבים 1-3 בנינו את שכבת הנתונים (DAL) של הפרויקט וכתבנו תכנית ראשית לבדיקת השכבה.

בשלב זה, שלב 4:

- עדכון **שכבת הנתונים (DAL)** לעבודה עם **תבניות עיצוב**.
- בניית **השכבה הלוגית (BL), נגדיר את החוזים שלה (BO/BlApi) ונממש אותם**
- בניית ממשק קונסול **לבדיקת השכבה הלוגית (BlTest)**

הנחיות לביצוע השלב והגשתו

- **שימו לב! ההנחיות במהלך המסמך מלווים בהסברים חשובים, להבנת כל מהלך ומהלך. הבנה זו תועיל לכם מאוד גם במהלך ההגנה בעל פה על הפרויקט וגם במהלך המבחן.**
- חובה על 2 השותפים להגיש במודל קישור על פי הנחיות ההגשה
- **שימו לב!** במידה ולא יהיה איזון מסוים בין אחוז הפעילות בגיט של 2 השותפים - הציון יהיה שונה בין 2 השותפים. דרך רישום הפעילות (push) בגיט נוכל לוודא ש 2 השותפים תורמים לתרגיל.
- **חובה** לקרוא את **מסמך התיאור הכללי של הפרויקט** לפני תחילת העבודה על השלב
- ההנחיות הנחשבות אפשרויות לתוספת תסומנה בכתום
- **חובה** לבצע את השלב באותו מאגר git והמאגר בענן ובאותו ה-Solution כמו בשלב 0 (המקדים)
- **חובה** להמשיך להקפיד על צורת כתיבת הקוד (הזחות, שמות וכו')
- **חובה** לתעד את כל הסוגים, המתודות, השדות, והתכונות בעזרת תיעוד מפורמט (///)

ראשי פרקים - שלב 4

- **עדכון שכבת הנתונים (DAL) לעבודה עם תבניות העיצוב (design patterns):**
  - Singleton
  - Simple factory Method
- **בניית שכבת הנתונים (BL = Business Layer):**
  - **BO** - נגדיר את **חוזה הנתונים** של השכבה הלוגית - הגדרת הישויות הלוגיות והתכונות (properties) שלהם
  - **BlApi** - נגדיר את **חוזה השירות** של השכבה הלוגית - הגדרת הממשקים (כנגד כל ישות לוגית ראשית), שיכילו מתודות לצורך גישה לכל ישות לוגית
  - נממש **ממשקי גישה** לישויות הלוגיות (מימוש מתודות הממשקים עבור כל סוג של ישות לוגית ראשית במחלקות מתאימות)
- **בניית תוכנית בדיקה ידנית (manual test program) של השכבה הלוגית**
  - **BlTest** - תכנית ראשית, ממשק קונסול טקסטואלי (CLI).

הנחיות פרטניות – שלב 4 - עדכון DAL לעבודה עם תבניות עיצוב

פרק 1 - הפיכת המחלקות הקיימות DalXml, DalList ל Singleton

נזכיר שמטרת תבנית העיצוב Singleton היא ליצור מחלקה שאפשר יהיה ליצור ממנה מופע (אובייקט) אחד לכל היותר. לכל היותר הכוונה שאם לא נעשה שימוש במחלקה זו אז לא נרצה ליצור אפילו אובייקט אחד ממנה. לכן לא נשתמש במחלקה סטטית כי ממנה תמיד מיוצר מופע אחד בלבד, ולפעמים לא נרצה שום מופע.

על מנת להפוך את מחלקת DalList לסינגלטון, בצעו את ההוראות הבאות על מחלקת DalList בפרויקט DalList:

1. הגדר את המחלקה כ **sealed**
2. החלף את הרשאת המחלקה **DalList** מ public ל **internal**
3. הוסף למחלקה תכונה **ציבורית סטטית** בשם **Instance** לקריאה בלבד (עם get בלבד), עם אתחול לאובייקט של המחלקה עצמה.
4. הוסיפו למחלקה בנאי **פרטי** (אם כבר היה קיים בנאי אז שנו את הרשאתו **לפרטית**).
5. כאפשרות לתוספת - ניתן לעשות סינגלטון שיהיה Thread Safe ועם Lazy Initialization מירבי - אבל רק אם אתם יודעים להסביר איך עשיתם ולמה זה Thread Safe ו-Lazy Initialization

```
namespace Dal;
using DalApi;


sealed internal class DalList : IDal
{
        public static IDal Instance { get; } = new DalList();
        private DalList() { }

        public IStudent Student { get; } = new StudentImplementation();
        public ICourse Course{ get; } = new CourseImplementation();
        public ILink Link { get; } = new LinkImplementation();
        public IConfig Config { get; } = new ConfigImplementation();

        public void ResetDB()
        {
                Student.DeleteAll();
                Course.DeleteAll();
                Link.DeleteAll();
                Config.Reset();
        }

}
```

על מנת להפוך את מחלקת DalXml לסינגלטון, בצעו את אותן ההוראות (1-4) על מחלקת DalXml בפרויקט DalXml.

שימו לב - בשלב זה נקבל שגיאת קומפילציה בתוכנית הראשית ב **DalTest** מכיוון ששינינו את הרשאת המחלקה ל internal

לאחר שנהפוך את כל השכבה לפקטורי, תיפתר השגיאה הזו.

```
static readonly IDal s_dal = new DalList(); //stage 2
//static readonly IDal s_dal = new DalXml(); //stage 3
```

פרק 2 - הגדרת מחלקת יצרן (Simple Factory) עבור שכבת הנתונים

2א. הקדמה - תבנית עיצוב מחלקת יצרן Simple Factory

נזכיר שמטרת תבנית העיצוב Simple Factory היא לאפשר לשכבה עליונה לייצר אובייקטים משכבה תחתונה אשר מממשים ממשק מוסכם, **מבלי להכיר את המחלקה המממשת**. השכבה התחתונה תגדיר "מפעל לייצור אובייקטים" (**Factory**), והייצור יעשה דרך מתודה (**Get**) שתחזיר עצם מאותחל מטיפוס הממשק.

אצלנו בפרויקט הנבנה ע"פ מודל השכבות:

- השכבה העליונה היא **BL** (שאותה נבנה בהמשך מסמך זה) והשכבה התחתונה היא **DAL**

- **ובהמשך,** השכבה העליונה תהיה **PL** והשכבה התחתונה שלה היא **BL**

כלומר את מחלקת היצרן (Factory) **נגדיר גם בשכבה DAL וגם בשכבה BL**, כי שתיהן משמשות כ "שכבה תחתונה".

נרצה שמפעל הייצור (Factory) במתודה Get, יקרא בזמן ריצה את שם המחלקה שאותה הוא צריך לייצר בשכבת הנתונים מקובץ קונפיגורציה. מ2 סיבות:

- כדי שיוחלט בזמן ריצה עם איזה מימוש של שכבת נתונים בחרנו לעבוד - עם רשימות בזכרון (List) או עם קבצי XML. ולא יהיה צורך לקמפל את שכבת הנתונים (DalFacade) כל פעם שרוצים להחליף מחלקה מממשת.
- בעיה טכנית - לא ניתן להגדיר רפרנס מעגלי. כרגע יש כבר רפרנס מכיוון DalXml/DalList לכיוון DalFacade ולא ניתן להגדיר רפרנס גם בכיוון ההפוך.

לצורך כך, בפרק הנוכחי:

- נגדיר קובץ קונפיגורציה בשם **dal-config.xml** שבו נציין את המחלקה שנרצה לעבוד איתה.
- נגדיר מחלקה בשם **DalConfig** שיודעת לקרוא את הקובץ בזמן ריצה ולקבל את שם המחלקה
- נגדיר מחלקה בשם **Factory** שתיעזר במחלקת **DalConfig** כדי לטעון ולאתחל את המחלקה הנכונה שאיתה נרצה לעבוד.

2ב. יצירת קובץ קונפיגורציה לקביעת סוג המימוש, בשם dal-config.xml

המטרה: הוספת לספריה **xml**, קובץ קונפיגורציה שדרכו נוכל לקבוע בכל ריצה **באיזה מהפרויקטים נרצה להשתמש כדי לגשת לבסיס הנתונים.** פרויקט DalList או פרויקט DalXml.

1. **בסייר הקבצים (File Explorer),** גשו לספריה xml
2. תחת ספריה זו, צרו קובץ חדש בשם **dal-config.xml**. לחצו על עכבר ימני ואז: **New → Text Document**, שנו את שם הקובץ כנדרש.
3. שימו לב שהסיומת של הקובץ היא **xml** בלבד ולא **xml.txt** - ודאו זאת דרך סייר הקבצים שלכם - זה ממש חשוב!
4. לאחר מכן, **גשו לויז'ואל סטודיו,** עמדו עם עכבר ימני על הספריה **xml** שיצרתם זה עתה בויז'ואל סטודיו
5. בתפריט ההקשר שנפתח, בחרו **Add → Existing Item**
6. בחרו להוסיף את הקובץ הקיים שזה עתה יצרתם: xml/**dal-config.xml**
7. **פתחו את הקובץ דרך הויז'ואל סטודיו** והעתיקו את הטקסט הבא להיות תוכן הקובץ עצמו:

```
<?xml version="1.0" encoding="utf-8"?>
<config>
  <dal>list</dal>
  <dal-packages>
        <list>DalList</list>
        <xml>DalXml</xml>
  </dal-packages>
</config>
```

8. בצעו שמירה (**Save**) של הקובץ

הסבר לגבי תוכן הקובץ dal-config.xml (עליכם לדעת להסביר את תוכנו):

- האלמנט הפנימי הראשון נקרא `<dal>` ובתוכו נכתוב שם של אלמנט אחר שקיים בקובץ, אותו האלמנט שמכיל את שם הפרויקט שבו נרצה להשתמש בריצה הנוכחית כדי לגשת לבסיס הנתונים.
- האלמנט הפנימי השני נקרא `<dal-packages>` והוא מכיל רשימה של חבילות (packages) אפשריות של סוגי בסיסי נתונים. בסיס נתונים מסוג list ובסיס נתונים מסוג קבצי xml.

למשל, בטקסט הקודם האלמנט `<dal>` מכיל את המילה **list**, כדי לציין שבתוך האלמנט שנקרא **<list>** נמצא שם הפרויקט שאיתו נרצה לעבוד בריצה הנוכחית (**DalList**).

ולמשל, בטקסט הבא נרצה לעבוד עם הפרויקט **DalXml** כך:

```
<?xml version="1.0" encoding="utf-8"?>
<config>
  <dal>xml</dal>
  <dal-packages>
        <list>DalList</list>
        <xml>DalXml</xml>
```

```
  </dal-packages>
</config>
```

ג2. הוספת מחלקת DalConfig - לקריאת קובץ הקונפיגורציה

הכנו עבורכם מחלקת עזר בשם **DalConfig** שיודעת לקרוא בזמן ריצה את הנתונים מקובץ הקונפיגורציה **dal-config.xml**.

בפרויקט DalFacade, הוסיפו תחת ספריה **DalApi** מחלקה חדשה בשם **DalConfig**. עם הקוד המוכן הבא:

שימו לב - מיד נסביר את הקוד, עליכם להבין מה עושה הקוד למרות שאתם מקבלים אותו מוכן.

```
namespace DalApi;
using System.Xml.Linq;

static class DalConfig
{
   /// <summary>
   /// internal PDS class
   /// </summary>
   internal record DalImplementation
   (string Package,   // package/dll name
    string Namespace, // namespace where DAL implementation class is contained in
    string Class        // DAL implementation class name
   );

   internal static string s_dalName;
   internal static Dictionary<string, DalImplementation> s_dalPackages;

   static DalConfig()
   {
      XElement dalConfig = XElement.Load(@"..\xml\dal-config.xml") ??
                        throw new DalConfigException("dal-config.xml file is not found");

      s_dalName =
         dalConfig.Element("dal")?.Value ?? throw new DalConfigException("<dal> element is missing");

      var packages = dalConfig.Element("dal-packages")?.Elements() ??
                        throw new DalConfigException("<dal-packages> element is missing");
      s_dalPackages = (from item in packages
                        let pkg = item.Value
                        let ns = item.Attribute("namespace")?.Value ?? "Dal"
                        let cls = item.Attribute("class")?.Value ?? pkg
                        select (item.Name, new DalImplementation(pkg, ns, cls))
                        ).ToDictionary(p => "" + p.Name, p => p.Item2);
   }
}

[Serializable]
public class DalConfigException : Exception
{
        public DalConfigException(string msg) : base(msg) { }
        public DalConfigException(string msg, Exception ex) : base(msg, ex) { }
}
```

הסבר הקוד של המחלקה DalConfig:

- מרחב שמות System.Xml.Linq כולל כלים לטיפול בקבצי xml וב-DOM XML.
- מתודה **XElement.Load** טוענת קובץ xml, מבצעת ניתוח שלו (parsing) ובונה עץ DOM (של Document Object Model) - עץ אובייקטים מטיפוס **XElement** ומחזירה את האובייקט של אלמנט השורש של קובץ ה xml.
- אובייקט מטיפוס **XElement** מכיל מידע מנותח על אלמנט כלשהו של xml.
- מתודה **Element("dal")** מביאה אובייקט של אלמנט עם שם **"dal"** מתוך העץ של DOM, והתכונה **Value** מביאה את הערך שבתוך האלמנט. כלומר:
  - אם האלמנט dal בקובץ נראה כך: **<dal>list</dal>**, אזי המחרוזת שתחזור לתוך s_dalName תהיה **list**
  - ואם האלמנט dal בקובץ נראה כך: **<dal>xml</dal>**, אזי המחרוזת שתחזור לתוך s_dalName תהיה **xml**
- זימון של **Element("dal-packages")?.Elements()** מחזיר אוסף של כל האלמנטים שהם "בניו" של אלמנט בשם **"dal-packages"** בעץ של DOM, זאת אומרת כל תת-האלמנטים של אלמנט **<dal-packages>**. כלומר:
  - `<list>DalList</list>`
  - `<xml>DalXml</xml>`
- מתודה **ToDictionary(p => "" + p.Name, p => p.Item2)** בונה ומחזירה טבלה מגובבת [hash table] ("מילון") ששם תת-האלמנט הוא מפתח גיבוב, וערך האלמנט הינו הערך המֻצָמָד למפתח הגיבוב, ומהקובץ שלנו נקבל (בעזרת רישום בפורמט דמוי json שהוא יותר אינטואיטיבי מ-xml להצגת אובייקטים):

```
{
   "list": "{DalImplementation { Package = DalList, Namespace = Dal, Class = DalList }}",
   "xml": "{DalImplementation { Package = DalXml, Namespace = Dal, Class = DalXml }}"
```

}

ד2. הוספת מחלקת Factory - לטעינת מחלקת המימוש שנקבעה בקובץ הקונפיגורציה

הכנו עבורכם מחלקת יצרן בשם **Factory** שתיעזר במחלקת **DalConfig** כדי לטעון ולאתחל את המחלקה הנכונה שאיתה נרצה לממש את שכבת הנתונים.

- מחלקת הייצור (**Factory**) תהיה **ציבורית וסטטית** והיא תהיה האחראית הבלעדית ליצירת האובייקטים שמממשים את הממשק.
  - המחלקה תהיה **ציבורית** על מנת שניתן יהיה לגשת אליה מהשכבה שמעליה
  - המחלקה תהיה **סטטית** על מנת ל"הכריח" את המתודות שלה להיות סטטיות - כדי לחסוך במופעים
- מחלקת הייצור תספק מתודה (**Get**) מייצרת **סטטית וציבורית** שתקבל פרמטר שבעזרתו היא תדע איזה טיפוס של אובייקט לייצר. ואם לא קיבלה פרמטר תייצר אובייקט ברירת מחדל שהיא מחליטה עליו.
  - המתודה תהיה **סטטית** על מנת לחסוך בייצור מופע של המחלקה (יתרון משמעותי אך לא חובה)
  - המתודה תהיה **ציבורית** על מנת שניתן יהיה לגשת אליה מהשכבה שמעליה - לייצר אובייקטים דרכה

בפרויקט DalFacade, הוסיפו תחת ספריה **DalApi** מחלקה חדשה בשם **Factory**. עם הקוד המוכן הבא:

שימו לב - מיד נסביר את הקוד, עליכם להבין מה עושה הקוד למרות שאתם מקבלים אותו מוכן.

```
namespace DalApi;
public static class Factory
{
        public static IDal Get
        {
        get
        {
        string dalType = DalApi.DalConfig.s_dalName ?? throw new DalConfigException($"DAL name is not
extracted from the configuration");
        DalApi.DalConfig.DalImplementation dal = DalApi.DalConfig.s_dalPackages[dalType] ?? throw new
DalConfigException($"Package for {dalType} is not found in packages list in dal-config.xml");

        try { System.Reflection.Assembly.Load(dal.Package ?? throw new DalConfigException($"Package
{dal.Package} is null")); }
        catch (Exception ex) { throw new DalConfigException($"Failed to load {dal.Package}.dll
package", ex); }

        Type type = Type.GetType($"{dal.Namespace}.{dal.Class}, {dal.Package}") ??
                throw new DalConfigException($"Class {dal.Namespace}.{dal.Class} was not found in
{dal.Package}.dll");

        return type.GetProperty("Instance", System.Reflection.BindingFlags.Public |
System.Reflection.BindingFlags.Static)?.GetValue(null) as IDal ??
                throw new DalConfigException($"Class {dal.Class} is not a singleton or wrong property
name for Instance");
        }
        }
}
```

הסבר הקוד של המחלקה Factory:

המחלקה מכילה מתודה בשם **Get שמייצרת/מחזירה אתחול של המחלקה המתאימה DalList או DalXml**, ע"פ מה שכתוב בקובץ הקונפיגורציה **dal-config.xml** שאותו היא יודעת לקרוא בעזרת מחלקת **DalConfig**.

המתודה Get:

- ניגש לשדה s_dalName של מחלקת DalConfig שהכנו ונקבל ממנו את שם האלמנט שמכיל את שם המחלקה שנרצה לממש
  - כלומר dalType יכיל בסופו של דבר את המחרוזת "xml" או "list".

- ניגש לטבלת גיבוב של מחלקת DalConfig שהכנו בסעיף הקודם ונפנה דרך אינדקס מסוג מחרוזת, למשתנה מסוג טבלה מגובבת ("מילון") **s_dalPackages[dalType]** שיחזיר לנו לתוך המשתנה **dal**, אובייקט מסוג **DalImplementation** שתואם למפתח שבאינדקס
- **dal.Package** יכיל את שם ה dll שמכיל את מחלקת המימוש שצריכה להיטען (**DalXml.dll or DalList.dll**)
- מתודה **Assembly.Load** תטען את המודול (קובץ) ***.dll** הנבחר
- הזימון הבא:

```
Type.GetType($"{dal.Namespace}.{dal.Class}, {dal.Package}")
```

מחזיר **reflection** (מטא-דטא) של הטיפוס **Dal.DalList** ממודול **DalList.dll** או של הטיפוס **Dal.DalXml** ממודול **DalXml.dll**

- הזימון **type.GetProperty** מחזיר מטא דטא של התכונה **"Instance"** של המחלקה DalXml/DalList שחייבת להיות סטטית ועם הרשאה public
- הזימון של **GetValue(null)** על המטא דטא של התכונה **"Instance"** מחזיר ערך של התכונה הסטטית הזו (שזהו אמור להיות המופע [היחיד] של מחלקת הסינגלטון **DalList/DalXml** שלנו)

פרק 3 - בדיקה חוזרת של שכבת הנתונים המעודכנת בעזרת הפרויקט הקיים DalTest

הריצו ובדקו שכל השינויים שעשינו עד כאן לשכבת נתונים (DAL), עובדים כראוי. שתבניות העיצוב מתפקדות כראוי, וש- 2 מחלקות המימוש השונות נטענות בזמן ריצה על פי קובץ הקונפיגורציה.

3א. עדכון DalTest לעבוד עם שכבת ה DAL המעודכנת

1. **בפרויקט DalTest, במחלקה Initialization, במתודה Do:**
   a. הסירו **מהגדרת המתודה Do** את הפרמטר המתקבל מסוג IDal (כלומר המתודה Do לא תקבל אף פרמטר)

```
//public static void Do(IDal dal) //stage 2
public static void Do() //stage 4
```

   b. **בגוף המתודה Do,** שנו את האתחול של השדה s_dal כך שבמקום שיאותחל בפרמטר שהוסר, הוא יאותחל לעבודה עם המחלקה התומכת בתבנית עיצוב Factory שמאתחלת את המחלקה המתאימה לפי קובץ הקונפיגורציה:

```
//s_dal = dal ?? throw new NullReferenceException("DAL object can not be null!"); //stage 2
s_dal = DalApi.Factory.Get; //stage 4
```

2. בפרויקט DalTest, במחלקה הראשית Program:
   a. שנו את **סוג האובייקט המאותחל עבור השדה IDal s_dal**. כך שבמקום שאתחול השדה s_dal יהיה עם שם מחלקה מפורש, אז השדה יאותחל לעבודה עם המחלקה התומכת בתבנית עיצוב Factory שמאתחלת את המחלקה המתאימה לפי קובץ הקונפיגורציה:

```
//static readonly IDal s_dal = new DalList(); //stage 2
//static readonly IDal s_dal = new DalXml(); //stage 3
static readonly IDal s_dal = Factory.Get; //stage 4
```

   b. בגוף המתודה Main, עדכנו את **הקריאה למתודה Initialization.Do**, להיקרא ללא פרמטר:

```
//Initialization.Do(s_dal); //stage 2
Initialization.Do(); //stage 4
```

3ב. הרצת הפרויקט כולו ב2 מצבי המימוש

כעת - ניתן לבדוק ששכבת הנתונים ממשיכה לעבוד כראוי, עם 2 צורות המימוש:

1. עדכנו את קובץ הקונפיגורציה **dal-config.xml** כך שיעבוד עם **DalList** (כתבו בתוך האלמנט `<dal>` את המילה list)
2. ואז הריצו את הפרויקט דרך DalTest ובדקו שהכל עובד כראוי.
3. לאחר מכן, עדכנו את קובץ הקונפיגורציה **dal-config.xml** כך שיעבוד עם **DalXml** (כתבו בתוך האלמנט `<dal>` את המילה xml)
4. ואז הריצו שוב את הפרויקט דרך DalTest ובדקו שהכל עובד כראוי.

הנחיות פרטניות - שלב 4 - בניית שכבת הנתונים BL

הקדמה - שכבת הנתונים BL

השכבה הלוגית, המכונה **BL = Business Layer**, מנהלת את כל הלוגיקה של המערכת שלנו. כל הכללים הלוגיים של המערכת ממומשים בה. השכבה תתקשר כלפי מטה עם שכבת נתונים (DAL) כדי לאחזר את הנתונים ולעדכנם. השכבה גם תתקשר עם שכבת התצוגה כדי לעבד את הקלט ולספק נתונים של קלט עבור ממשק המשתמש.

השכבה הלוגית תהיה אחראית על:

- הגדרת טיפוסים של ישויות לוגיות על פי צורכי התצוגה (ממשק המשתמש)
- הלוגיקה שמאחורי נתונים:
  - בדיקת תקינות והתאמת נתוני הקלט שהגיעו מהתצוגה
  - בדיקת התאמת נתוני קלט ע"פ המידע בשכבת הנתונים
  - שלמות והתאמה של כל הנתונים המאוחסנים בשכבת נתונים
  - ביצוע חישובים והפקת תוצאות מהנתונים היבשים שהגיעו מהשכבה שמתחתיה.

בפרק הבא, נוסיף 2 פרויקטים ל Solution:

- **פרויקט בשם BL מסוג Class Library שיהווה את השכבה הלוגית (BL = Business Layer) ויכיל:**
  - **BO** - נגדיר את **חוזה הנתונים** של השכבה הלוגית - הגדרת **ישויות נתונים לוגיות** והתכונות (properties) שלהם
  - **BlApi** - נגדיר את **חוזה השירות (ממשקים)** של השכבה הלוגית - הגדרת **ישויות שירות לוגיות**, שיכילו מתודות לצורך הפעלה של פעולות על ישויות נתונים לוגיות
  - נממש את **חוזי השירות הלוגיים -** מימוש מתודות הממשקים.
- **פרויקט בשם BlTest מסוג Console Application שיהוו תוכנית בדיקה ידנית (manual test program) של השכבה הלוגית**

פרק 4 - יצירת השכבה הלוגית - הוספת 2 פרויקטים BL ו- BlTest

נוסיף כעת למודל השכבות את השכבה האמצעית (השכבה הלוגית) - פרויקט בשם **BL** מסוג **Class Library**.

ולאחר מכן, נוסיף גם פרויקט בשם **BlTest** מסוג **Console Application** לצורך בדיקת השכבה הלוגית.

4א. הוספת פרויקט BL מסוג Class Library

1. הוסיפו ל **Solution** פרויקט חדש בשם **BL** מסוג **Class Library**.
2. לאחר שייפתח פרויקט חדש, לחצו לחיצה כפולה על שם הפרויקט ב-Solution Explorer
3. ייפתח לעריכה קובץ **לניהול המאפיינים של הפרויקט**, הוסף לקובץ את השורות הבאות (המודגשות בצהוב) בסוף האלמנט **PropertyGroup**:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <BaseOutputPath>$(SolutionDir)\bin\</BaseOutputPath>
    <OutputPath>$(SolutionDir)\bin\</OutputPath>
    <AppendTargetFrameworkToOutputPath>false</AppendTargetFrameworkToOutputPath>
    <AppendRuntimeIdentifierToOutputPath>false</AppendRuntimeIdentifierToOutputPath>
    <UseCommonOutputDirectory>true</UseCommonOutputDirectory>
    <RootNamespace />
  </PropertyGroup>
</Project>
```

4. בצעו שמירה (**Save**) של הקובץ
5. **הוסיפו references מכיוון פרויקט BL לעבר פרויקט DalFacade, פרויקט DalList, ופרויקט DalXml:**
   a. עמדו עם עכבר ימני על הפרויקט BL ואז **Add → Project Reference**
   b. בחלון שיפתח סמנו את 4 הפרויקטים DalFacade, DalList, DalXml, DalTest, ואז לחצו **OK**
6. ודאו ב**קובץ המאפיינים של הפרויקט** שהטקסט שלו התעדכן לאחר הוספת הרפרנס:

```
<ItemGroup>
```

```
    <ProjectReference Include="..\DalFacade\DalFacade.csproj"/>
    <ProjectReference Include="..\DalList\DalList.csproj"/>
    <ProjectReference Include="..\DalXml\DalXml.csproj"/>
    <ProjectReference Include="..\DalTest\DalTest.csproj"/>
</ItemGroup>
```

נזכיר את המשמעות של רפרנס מכיוון פרויקט X לכיוון פרויקט Y:

a. בזכות הרפרנס, פרויקט X יכיר את ההגדרות שנמצאות בפרויקט Y **ולא ההיפך!**
b. וכן, כאשר נבנה/נקמפל את פרויקט X **אז באופן אוטומטי הוא קודם כל יבנה/יקמפל את הפרויקט שהוא תלוי בו** (Y).

את הרפרנס מפרויקט BL לפרויקט DalFacade, DalTest הגדרנו מהסיבות a, b.
את הרפרנס מפרויקט BL לפרויקטים DalList, DalXml הגדרנו רק מסיבה b. **זהו רפרנס מאולץ**.שנועד רק כדי לגרום לפרויקטים הללו להיבנות באופן אוטומטי כאשר בונים את פרויקט BL, וקיומם באותה ספריית הרצה.

4ב. הוספת פרויקט BlTest מסוג Console Application

1. הוסיפו ל **Solution** פרויקט חדש בשם **BlTest** מסוג **Console Application**.
2. לאחר שייפתח פרויקט חדש תחת הפתרון, לחצו לחיצה כפולה על שם הפרויקט ב-Solution Explorer
3. ייפתח לעריכה קובץ **לניהול המאפיינים של הפרויקט**. הוסף לקובץ את השורות הבאות (המודגשות בצהוב) בסוף האלמנט **PropertyGroup**:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <BaseOutputPath>$(SolutionDir)\bin\</BaseOutputPath>
    <OutputPath>$(SolutionDir)\bin\</OutputPath>
    <AppendTargetFrameworkToOutputPath>false</AppendTargetFrameworkToOutputPath>
    <AppendRuntimeIdentifierToOutputPath>false</AppendRuntimeIdentifierToOutputPath>
    <UseCommonOutputDirectory>true</UseCommonOutputDirectory>
  </PropertyGroup>
</Project>
```

4. בצעו שמירה (**Save**) של הקובץ
5. הוסיפו **reference** מכיוון פרויקט BlTest לעבר פרויקט BL
6. ודאו ב**קובץ המאפיינים של הפרויקט** שהטקסט שלו התעדכן לאחר הוספת הרפרנס:

```
<ItemGroup>
    <ProjectReference Include="..\BL\BL.csproj"/>
</ItemGroup>
```

7. בסיום שלב זה, לאחר שנבנה את השכבה הלוגית (פרויקט BL), תרצו לסמן את הפרויקט BlTest **כפרויקט ההתחלתי של ה-Solution**, הפרויקט שירוץ כשלוחצים על כפתור הPLAY הירוק:
   a. עמדו עם עכבר ימני על הפרויקט BlTest
   b. ואז לחצו על **Set as Startup Project**

פרק 5 - יצירת 4 תתי ספריות תחת פרויקט BL

בשלבים 1-3, כשעיצבנו את הארכיטקטורה של שכבת הנתונים (DAL), הפרדנו את החוזים (DO/DalApi) והמימוש שלהם לפרויקטים נפרדים. את החוזים הגדרנו בפרויקט DalFacade ואת המימוש שלהם הגדרנו בפרויקט DalList ובפרויקט DalXml, כאשר כל אחד מימש בצורה שונה.

לעומת זאת, כעת בשלב 4, הארכיטקטורה של השכבה הלוגית (BL), נגדיר את החוזים (BO/BlApi) והמימוש **באותו פרויקט בשם BL**.

לצורך כך, הוסיפו לפרויקט BL את 4 תתי-הספריות הבאות:

1. תת-ספריה בשם **BO** - עבור הישויות הלוגיות
2. תת-ספריה בשם **BlApi** - עבור ממשקי השכבה הלוגית
3. תת-ספריה בשם **BlImplementation** - עבור **המימוש** של מתודות הממשק של השכבה הלוגית
4. תת-ספריה בשם **Helpers** - עבור מחלקות עזר בהרשאת internal

(**תזכורת ליצירת תת-ספרייה:** עמדו עם עכבר ימני על פרויקט BL ואז לחצו על Add → New Folder וקראו לספריה בשמה)

פרק 6 - הגדרת ישויות הנתונים הלוגיות (חוזי נתונים - מחלקות) - BO

את רשימת ישויות הנתונים הלוגיות (ישויות BO) הנדרשות בפרויקט של שנה"ל תוכלו למצוא במסמך נפרד שמתאר את הפרויקט באופן כללי. שם תוכלו גם לראות את התכונות הנדרשות עבור כל ישות נתונים לוגית.

בשלב 5 והלאה, כשתתחילו לבנות את שכבת התצוגה, ייתכן שתרצו לשנות\לעדכן את הגדרת ישויות הנתונים הלוגיות בהתאם לצרכי התצוגה.

6א. הקדמה - ישויות נתונים לוגיות (ישות BO)

- ישויות הנתונים הלוגיות (ישויות BO) יעברו בין שכבת התצוגה העליונה לבין השכבה הלוגית שמתחתיה. הם יעברו ב2 הכיוונים. ישות נתונים לוגית מוגדרת כך שהיא תארוז מידע עבור הצרכים של שכבת התצוגה (המסכים השונים).
- **ישויות הנתונים הלוגיות (BO)** מבוססות על **ישויות הנתונים (DO)** אך לא זהות להן. הן תוצר של הסקת מסקנות מהקשרים שבין ישויות הנתונים (DO) ושאילתות על המידע הכולל בבסיס הנתונים.
- **יש הרבה יותר** ישויות נתונים לוגיות (BO) מישויות נתונים (DO).
- **כל ישות נתונים לוגית מוגדרת כ- PDS חלקי:**
  - **שטוחה**: מכילות **רק תכונות (properties)** ללא קוד, לא מכילות מתודות (אפילו לא בנאי), למעט העמסה של ToString לצורכי debug בלבד.
  - יכולה להכיל **תכונות** מטיפוס value type
  - יכולה להכיל **תכונות** מטיפוס reference type
  - יכולה להכיל **תכונות** מטיפוס של **ישות נתונים לוגית (BO)** אחרת
    - ולא מטיפוס של ישות נתונים (DO)
  - יכולה להכיל **תכונות** מסוג רשימה/מערך כלשהו
  - יכולה להכיל **תכונות מטיפוס רשימה/מערך** של ישות נתונים לוגית אחרת List<T>
    - ולא מטיפוס רשימה/מערך של ישויות נתונים (DO)
  - יכולה להכיל **תכונות מטיפוס של ישות נתונים לוגית אחרת**
  - יכולה **לרשת** מישות נתונים לוגית אחרת
- חלק **מישויות הנתונים הלוגיות (BO)** מכילות **מידע חלקי (מצומצם)** עבור **ישות נתונים (DO) "מקבילה"**
  - לדוגמא: הצגת רשימה של ישות נתונים לוגית כלשהי - המידע שיוצג ברשימה עבור כל ישות נתונים הוא מידע חלקי (מצומצם) אשר לחיצה עליו תוביל להצגת המידע כולו.
- **השכבה הלוגית לא שומרת אצלה שום נתונים ולא צריכה להגן עליהם במעבר בין השכבות.** כלומר, המופעים של ישויות הנתונים הלוגיות (BO) שנוצרים בשכבה הלוגית **לא נשמרים בשום בסיס נתונים,** אלא מגיעים מוכנים משכבת התצוגה כפרמטרים של מתודות.
  - בתוך המתודות מרכיבים מהם אובייקטים מסוג ישויות נתונים (DO). ואותם מעבירים כלפי מטה לשכבת הנתונים.
  - **לכן, ישויות הנתונים הלוגיות BO יוגדרו כ reference types מסוג class.**
    - לא לצריך להגן עליהן משינויים, הן לא חייבות להיות **immutable**, ולכן יכולות להיות class רגיל
    - **בניגוד לישויות DO בשכבת הנתונים שנשמרות בבסיס הנתונים**, ואנו צריכים להגן עליהן משינויים במעבר בין השכבות. לכן בחרנו להגדיר ישויות **DO** כ record class שמאפשר לנו להגדיר ישויות **immutable** בצורה נוחה וקצרה.

6ב. הוספת קובץ אנומרציות לספריה BO

מלבד קבצי הגדרת ישויות הנתונים הלוגיות, בספריה זו גם נשמור גם את **קובץ האנומרציות** הנחוצות בהגדרת הישויות הלוגיות.

1. הוסיפו קובץ (עכבר ימני על הספריה **BO**) אנומרציות ריק בשם **Enums.cs.**
2. **רוקנו** את הקובץ כך שתישאר בו רק השורה:

```
namespace BO;
```

שימו לב - הגדרת האנומרציות בשכבה BL נפרדת מהאנומרציות בשכבת DAL, גם אם חלקן יהיו זהות. בהמשך פרק זה, כאשר תגדירו את ישויות הנתונים הלוגיות עצמן, ינבע מהן **צורך בהגדרות של אנומרציות** שאותן תגדירו בקובץ שיצרתם כעת.

ג6. יצירת טיפוס חדש עבור כל ישות נתונים לוגית

כעת ניגש ליצירת טיפוס חדש **עבור כל ישות נתונים לוגית (ישות BO)** נדרשת. כל ישויות הנתונים הלוגיות, ישמרו תחת תת-ספריה **BO**.

1. **עבור כל אחת מישויות הנתונים הלוגיות BO הנדרשות במסמך הכללי של הפרויקט,** עמדו עם עכבר ימני על הספריה **BO** והוסיפו **מחלקה** עבור הישות ע"פ הכללים הבאים.

הנחיות כלליות להגדרת ישות נתונים לוגית ותכונותיה:

- כל ישות נתונים לוגית תוגדר כ **class**, **בקובץ נפרד** תחת תת-ספריה **BO** ולכן גם תחת מרחב שמות **BO**
- שם המחלקה יהיה זהה לשם ישות הנתונים הלוגית ויתחיל באות גדולה
- הרשאת המחלקה תהיה **public**
- כל התכונות של כל ישות נתונים לוגית יהיו **properties ציבוריים** ולא שדות.
  - יש לחשוב לגבי כל תכונה האם היא צריכה להיות **nullable** או לא. ע"פ אותם שיקולים משלב 1.
  - תכונה שהיא מסוג Enum, **יש להוסיף עבורה הגדרה חדשה של אנומרציה** בקובץ האנומרציות של השכבה הלוגית (יצרנו בסעיף קודם Enums.cs)
    - אין להשתמש באנומרציות של שכבת DAL אלא להגדיר אנומרציות חדשות עבור שכבת ה BL. גם אם חלקו חוזרות על עצמו.
  - כל תכונה תוגדר **ללא אתחול** בהצהרה.
  - תכונה שאמורה לקבל ערך רק באתחול (כגון מספר מזהה) תוגדר כך:

```
{ get; init; }
```

  - שאר התכונות תוגדרנה כך:

```
{ get; set; }
```

- **ישות נתונים לוגית (BO) צריכה להיות "שטוחה חלקית" (PDS חלקי - ראה הסבר בפרק זה למעלה)**
  - בניגוד לישות נתונים (DO) שחייבת להיות שטוחה (PDS)
- **אסור להוסיף למחלקה כל לוגיקה חוץ מדריסה של ToString שנועדה לצרכי debug**
  - כלומר, אסור להוסיף: בנאים מכל סוג, הורס, או מתודות כלשהן.
  - **המימוש של ToString עבור כל ישות יהיה בעזרת מתודת הרחבה ובשיטת Reflection**. ראו פירוט כאן: ג7. הגדרת מחלקה פנימית בשם
  - עבור מתודות עזר כלליות בשכבה הלוגית
- כל ישות והתכונות שלה **יתועדו** בעזרת ההערה המפורמטת ///. כמו בשלב 1.

דוגמאות לישויות נתונים לוגיות אפשריות, למערכת הסטודנטים-קורסים שבעזרתה אנחנו מדגימים לכל אורך השלבים, ניתן לראות כאן: נספח 1 - דוגמאות לישויות נתונים לוגיות

ד6. יצירת טיפוס חדש Config שיכיל את כל משתני התצורה

כעת ניגש ליצירת טיפוס חדש בשם Config **שיכיל תכונה עבור כל משתנה תצורה** שצריך להיות חשוף כלפי מעלה (שכבת התצוגה). משתני תצורה שהם לשימוש פנימי בשכבת נתונים (כמו מספרים רצים) לא יתווספו כאן. הטיפוס Config ישמרו תחת תת-ספריה **BO**.

1. עמדו עם עכבר ימני על הספריה BO והוסיפו מחלקה ציבורית בשם Config
2. עבור כל משתנה תצורה שצריך להיות חשוף כלפי שכבת התצוגה, הוסיפו למחלקה תכונה מתאימה.

```
namespace BO;
public class Config
{
        public int MaxRange { get; set; }
        //TO_DO: //stage 4
        //add props from DalApi.IConfig that we want to show/change in PL
        //...
}
```

פרק 7 - הגדרה ומימוש של מחלקות עזר סטטיות לשימוש פנימי בשכבה הלוגית

הערות חשובות המסבירות את הקשר בין 3 הפרקים 7,8,9:

- **ב 2 הפרקים הבאים(פרק 8 ופרק 9)**, נגדיר ממשק עבור כל **ישות שירות לוגית (פרק 8)** ונממש את **מתודות הממשקים (פרק 9)** תוך **שימוש פנימי במתודות העזר הסטטיות** שנגדיר ונממש כעת בפרק הנוכחי **(פרק 7)**
- **בפרק הנוכחי (פרק 7)**, נגדיר ונממש **מחלקות עזר סטטיות לצורך שימוש פנימי (בהרשאת internal)** במימוש השכבה הלוגית**):**
  - נגדיר ונממש מחלקת עזר כללית בשם **Tools** לצורך **מתודות עזר סטטיות כלליות** לשימוש פנימי בשכבה הלוגית
  - נוסיף מחלקת עזר לצורך **ניהול לוגי של משתני התצורה ושעון המערכת** (מחלקה מוכנה בשם AdminManager. שכבר הגדרנו וממשנו עבורכם)
  - נגדיר ונממש מחלקות עזר לוגיות (**מחלקת עזר נפרדת עבור כל ישות נתונים)** לצורך **ניהול לוגי של ישויות הנתונים.**

7א. הגדרת מחלקה פנימית בשם Tools עבור מתודות עזר כלליות לשימוש פנימי בשכבה הלוגית

נרצה לייצר **קובץ עבור מתודות עזר** לשימושים כלליים (שימוש של כל הישויות ולא שימוש של ישות ספציפית).

מתודות העזר, יהיו מסוג:

- מתודות הרחבה סטטיות
- גנריות
- כלומר המתודות יתווספו באופן אוטומטי לכל טיפוס קיים, T.

לצורך כך, בפרוייקט BL:

1. הוסיפו מחלקה **סטטית** עם הרשאת **internal** בשם **Tools**, בקובץ נפרד.
   a. הקובץ (המחלקה) יתווסף תחת תת הספריה בשם **Helpers** (אם תת הספריה לא קיימת - צרו אותה תחת פרויקט BL)
2. בהמשך, כל פעם שתרגישו צורך בהוספת מתודת עזר, הוסיפו אותן בקובץ זה.

דוגמא למתודה שיכולה להיות לעזר בשכבת BL:

```
public static string ToStringProperty<T>(this T t)
```

הסבר - מתודת הרחבה סטטית בשם ToStringProperty אשר:

- **תחקור** כל טיפוס T בזמן ריצה בעזרת **Reflection**
- תייצר **מחרוזת** מכל התכונות שלו **שנחקרו** בזמן ריצה
- המתודה תזומן מתוך כל **העמסה** של **ToString** בכל **ישות BO**:

```
public override string ToString() => this.ToStringProperty();
```

מכיוון **שישויות BO הן שטוחות חלקית (PDS חלקי)** ויכולות להכיל תכונות שהם אוסף/רשימה, אז המתודה צריכה לדאוג **להיכנס לעומק התכונות הללו ולהחזיר מחרוזת שכוללת את איברי הרשימה** כחלק מערכי התכונות של הישות.

7ב. הוספת מחלקה סטטית פנימית לניהול לוגי של משתני תצורה ושעון המערכת

לצורך **ניהול לוגי פשוט יותר של משתני התצורה ושעון המערכת**, הכנו עבורכם קובץ עזר בשם **AdminManager.cs**, המכיל מחלקת עזר סטטית בהרשאת internal.

ניתן למצוא את הקובץ בקישור הבא : AdminManager.cs

1. דרך סייר הקבצים, העתיקו את הקובץ הפיזי **AdminManager.cs** תחת תת הספריה בשם **BL\Helpers** (אם תת הספריה לא קיימת - צרו אותה תחת פרויקט BL)
2. לאחר שהקובץ הנ"ל הועתק פיזית לספריה הנכונה, הוסיפו אותו לפרויקט תחת **BL\Helpers**

המחלקה מחולקת לאזורי קוד (regions) לפי שלבים. ומכילה קוד מוכן שהכנו עבורכם עבור שלבים 4 עד 7. **נסביר כעת רק את החלקים במחלקה הרלוונטיים לשלב 4. עבור שאר אזורי הקוד של המחלקה, הנוגעים לשלבים הבאים - נרחיב בכל שלב בנפרד.**

עליכם להשלים חלקים מסוימים בקובץ והם מסומנים בהערה:

```
//TO_DO: //stage 4
```

הסבר לגבי תוכן הקובץ, כיצד עליכם להשתמש בו במימוש השכבה הלוגית ומה עליכן להוסיף בו :

התכונה Now - מופיעה באזור בשם # region Stage 4-7 :

המחלקה מכילה את התכונה **Now** שמחזירה את ערכו הנוכחי של שעון המערכת כפי שנשמר בשכבת ה-DAL

כל גישה לשעון המערכת בשכבת ה BL (בשלב מימוש הממשקים בפרק 9) תהיה מותרת אך ורק דרך תכונה זו ולא ישירות דרך ה DAL.

התכונה מוגדרת כך:

```
internal static DateTime Now { get => _dal.Config.Clock; } //stage 4
```

ושימוש בה בשכבת ה BL יהיה כך:

```
AdminManager.Now
```

המתודה UpdateClock - מופיעה באזור בשם # region Stage 4-7 :

המחלקה מכילה את המתודה **UpdateClock** שמקבלת זמן חדש **ומעדכנת את ערכו של שעון המערכת לזמן החדש**.

בנוסף, המתודה **מזמנת מתודות תקופתיות** (אחת או יותר) שיהיו צריכות להיקרא בכל פעם שהזמן של השעון מתקדם. מתודות תקופתיות אלו **יוגדרו על ידכם בפרק הבא (פרק 9)**, בקוד שכאן למטה הבאנו דוגמא לזימון של מתודה תקופתית לעדכון סטודנטים.

סימנו עבורכם את המקום בו תצטרכו להוסיף בהמשך זימונים למתודות עזר תקופתיות בהערה:

```
//TO_DO: //stage 4
```

המתודה UpdateClock מוגדרת כך:

```
internal static void UpdateClock(DateTime newClock) //stage 4-7
{
        var oldClock = s_dal.Config.Clock; //stage 4
        s_dal.Config.Clock = newClock; //stage 4

        //Add calls here to any logic method that should be called periodically,
        //after each clock update
        //for example, Periodic students' updates:
        // - Go through all students to update properties that are affected by the clock update
        // - (students become not active after 5 years etc.)

        //TO_DO: //stage 4
        StudentManager.PeriodicStudentsUpdates(oldClock, newClock); //stage 4
        //...

        //Calling all the observers of clock update
        ClockUpdatedObservers?.Invoke(); //prepared for stage 5
}
```

ושימוש בה בשכבת ה BL בהמשך יהיה כך:

```
AdminManager.UpdateClock(AdminManager.Now);
AdminManager.UpdateClock(AdminManager.Now.AddDays(1));
```

המתודה GetConfig - מופיעה באזור בשם # region Stage 4-7 :

המחלקה מכילה את המתודה **GetConfig** שמייצרת ומחזירה אובייקט מטיפוס Config שמכיל את כל משתני התצורה הנצרכים ונלקחים מ s_dal.Config.

המתודה GetConfig מוגדרת כך:

עליכם להשלים בתוכה את כל ההשמות של משתני התצורה הרלוונטים לפרויקט שלכם.

```
internal static Config GetConfig() //stage 4
=> new Config()
{
        MaxRange = s_dal.Config.MaxRange
        //TO_DO: //stage 4
        //add an assignment for each configuration property
        //...
};
```

ושימוש בה בשכבת ה BL בהמשך מתוך AdminImplemenation יהיה כך:

```
public Config GetConfig() => AdminManager.GetConfig();
```

המתודה SetConfig - מופיעה באזור בשם # region Stage 4-7 :

המחלקה מכילה את המתודה **SetConfig** שמקבלת אובייקט מטיפוס Config שמכיל את כל משתני התצורה העדכניים ומעדכנת בעזרתו את כל משתני התצורה הנמצאים ב: s_dal.Config. שימו לב שכל אחד ממשתני התצורה מתעדכן רק במידה והוא שונה מהערך הנוכחי שלו.

המתודה SetConfig מוגדרת כך:

עליכם להשלים בתוכה את כל העדכונים למשתני התצורה הרלוונטים לפרויקט שלכם.

```
internal static void SetConfig(BO.Config configuration) //stage 4
{
        bool configChanged = false; // stage 5

        if (s_dal.Config.MaxRange != configuration.MaxRange) //stage 4
        {
                s_dal.Config.MaxRange = configuration.MaxRange;
                configChanged = true;
        }
        //TO_DO: //stage 4
        //add a condition+assignment for each configuration property
        //...

        //Calling all the observers of configuration update
        if (configChanged) // stage 5
                ConfigUpdatedObservers?.Invoke(); // stage 5
}
```

ושימוש בה בשכבת ה BL בהמשך מתוך AdminImplemenation יהיה כך:

```
public void SetConfig(BO.Config configuration)
{
        AdminManager.SetConfig(configuration);
}
```

המתודות ResetDB, InitializeDB - מופיעות באזור בשם # region Stage 4-7 :

מפעילות את אתחול בסיס הנתונים ואיפוסו על ידי קריאות למתודות שהכנו בשכבת ה DAL.

בנוסף, מעדכנות את השעון לזמן הנוכחי בהתאם ומעדכנות את ישות התצורה על עדכון השעון.

ג7. הגדרת מחלקת עזר סטטית פנימית עבור כל ישות נתונים

רגע לפני שנגדיר ונממש את ממשקי השירות הלוגיים, נצטרך להגדיר מחלקות עזר סטטיות (בהרשאת internal) שיכילו מתודות עזר סטטיות **לצורך שימוש פנימי** (יזומנו מתוך מתודות המימוש של הממשקים שנגדיר בפרק הבא).

עבור כל ישות נתונים (ישות DO - ולא ישות BO), נגדיר ונממש מחלקת עזר לוגית נפרדת לצורך **ניהול לוגי של ישות הנתונים. מחלקת העזר תכיל מתודות עזר סטטיות עבור:**

- קוד שחוזר על עצמו (יותר יעיל לקבצו למתודה נפרדת)
- קוד בעל לוגיקה מורכבת (יותר קריא להוציאו למתודה נפרדת)
- קוד לשימוש פנימי שאינו חלק מהממשק הציבורי, למשל:
  - **מתודה לצורך עדכון מצב תקופתי** - מתודה שמופעלת לאחר כל עדכון של שעון המערכת ובודקת האם יש תכונות של ישויות שצריכות להתעדכן בהתאם לקידום הזמן בשעון
  - **מתודה לצורך ביצוע סימולציה (הדמיה) כלשהי** - מתודה שתופעל כחלק מהסימולטור **שנבנה בשלב 7 של הפרויקט**. למשל מתודה שתדמה מצב של רישום לקורסים ונתינת ציונים במהלך הזמן המתקדם.

מתודות העזר שנממש כעת יממשו בעצם את כל הלוגיקה של המערכת שלנו, הן אלו שיבדקו נכונות לוגית, יפעילו שאילתות (חלקן מורכבות) לפני ביצוע של פעולות לכיוון שכבת נתונים (DAL) כדי לוודא את חוקיות הפעולה וכו'.

לצורך כך:

1. **עבור כל אחת מישויות הנתונים DO הנדרשות במסמך הכללי של הפרויקט**, עמדו עם עכבר ימני על הספריה **Helpers** והוסיפו **מחלקת עזר לוגית** עבור הישות ע"פ הכללים הבאים:
   a. כל מחלקת עזר לוגית תוגדר כ **internal static class**, **בקובץ נפרד** תחת תת-ספריה **Helpers** ולכן גם תחת מרחב שמות **Helpers**
   b. שם המחלקה יורכב משם ישות הנתונים בצירוף המילה **Manager**
      i. לדוגמא, עבור ישות נתונים DO.Student מחלקת העזר תיקרא StudentManager
   c. כל המתודות במחלקה יהיו internal static
   d. הוסיפו למחלקה שדה פרטי סטטי שיאפשר גישה כלפי מטה ל DAL:

```
private static IDal s_dal = Factory.Get; //stage 4
```

- **הערה חשובה: בסעיף הנוכחי, מן הסתם עוד לא תגדירו ותממשו את כל מתודות העזר. וייתכן אף שהמחלקה תישאר ריקה ממתדות בשלב זה. אך בהמשך (בפרק 9), תוך כדי מימוש מתודות הממשקים, תרגישו צורך להוסיף ולהגדיר מתודות עזר.**

הנחיות כלליות להגדרה ומימוש מתודות מחלקת עזר לוגית עבור ישות נתונים DO:

- **מתוך המימוש, תתבצענה פניות למתודות הממשקים של שכבת נתונים (DAL)**
  - כל מחלקה עזר מממשת תגדיר בתוכה שדה פרטי מסוג IDal שדרכו יפנו המתודות לממשקי ה DAL השונים ויפעילו מתודות מתאימות.
  - המתודות יפנו לשכבה התחתונה דרך ממשקי DAL, יקבלו ממנה אובייקטים של DO ויהפכו אותם לאובייקטים של BO שאותם יחזירו כערך חוזר של המתודה, כלפי מעלה לשכבת התצוגה PL.
  - ובכיוון ההפוך, המתודות יקבלו כפרמטרים אובייקטים של BO מהשכבה העליונה PL, יהפכו אותם לאובייקטים של DO שאותם ישלחו דרך ממשקי ה DAL כדי לתשאל/לעדכן את בסיס הנתונים.
- **במימוש מתודות העזר**, יש לעשות שימוש בתחביר LINQ בעזרת שתי האפשרויות שלמדנו למימוש:
  - בעזרת מתודות הרחבה המוגדרות במחלקת Enumerable (מרחיבות את האוספים שיורשים מ IEnumerable)
  - בעזרת תחביר שאילתות linqToObject
  - **עם זאת, חובה להשתמש ב 2 האפשרויות הנ"ל עם לפחות 4 מופעים מכל סוג!**
- **במימוש מתודות העזר**, אסור להשתמש בלולאת foreach **במקומות שבהן ניתן להשתמש בתחביר שאילתות** ו-\או במתודות הרחבה (רצוי בשרשור)
  - כלומר, רק אם אין אפשרות להשתמש בהן אז נאשר להשתמש בלולאת foreach
- **במימוש מתודות העזר**, חובה להשתמש בביטויי למבדה במקומות המתאימים.
- ב-linqToObject חובה להשתמש לפחות פעם אחת ב:

- let
- select new
- קיבוץ (grouping)
- מיון

- בשכבת BL אסור להחזיק אובייקטים או אוספי אובייקטים של ישויות מכל סוג (ישויות נתונים או ישויות לוגיות אפילו לא ישויות BO) אלא רק פעולות!! האובייקטים נשמרים ב שכבת ה DAL בסופו של דבר.

- המתודות יעטפו קריאות ל DAL בבלוק try/catch ויתפסו **חריגות DO**. חריגות DO ישלחו **כחריגות פנימיות בתוך חריגות BO חדשות** שיזרקו כלפי מעלה לתצוגה.

- **המלצה חמה** - אל תוסיפו בראש קובץ המימוש using namespace BO/DO, זאת על מנת שתאולצו לציין את מרחב השמות DO/DO לפני כל שם של טיפוס ישות לוגית או ישות נתונים. במתודות הללו עושים שימוש ב 2 סוגי הישויות והם בעלי שמות זהים ולכן זה יכול מאוד לבלבל. כך יקל עליכם להבדיל בין ישות לוגית לישות נתונים.
  - למשל: להבדיל בין BO.Student לבין DO.Student.

לפעמים, תוך כדי העבודה מרחב השמות נוסף אוטומטית, זה יכול לקרות כמה פעמים, **המלצה חמה** - תמיד להסיר אותו.

דוגמאות למתודות עזר לוגיות אפשריות, למערכת הסטודנטים-קורסים שבעזרתה אנחנו מדגימים לכל אורך השלבים, ניתן לראות כאן: נספח 2 - דוגמאות להגדרה ומימוש של מתודות עזר לוגיות

פרק 8 - הגדרת ישויות השירות הלוגיות (חוזי השירות - ממשקים) - BlApi

את רשימת ישויות השירות הלוגיות הנדרשות בפרויקט של שנה"ל תוכלו למצוא במסמך נפרד שמתאר את הפרויקט באופן כללי. שם תוכלו גם לראות את הפעולות (המתודות) הנדרשות עבור כל ישות שירות לוגית (ממשק).

מטרת הממשקים של השכבה הלוגית היא לרכז ולהצהיר על הפעולות/הפונקציונליות שהשכבה הלוגית מאפשרת עבור שכבת התצוגה שמעליה. הממשקים שנגדיר:

- **ישויות שירות לוגיות (ממשקים)**. אשר יכללו את המתודות הקשורות לאותה ישות שירות לוגית.
- **ממשק לוגי ראשי** בשם **IBl** שירכז את הגישה **לכל ישויות השירות הלוגיות (ממשקים)**

8א. הגדרת ישות שירות לוגית (ממשק)

כעת ניגש ליצירת טיפוס חדש **של ממשק לוגי** שנדרש עבור כל ישות שירות לוגית. כל הממשקים הלוגיים, ישמרו תחת תת-ספריה **BlApi**.

1. **עבור כל אחת מישויות השירות הלוגיות (כולל ממשק הניהול IAdmin), הנדרשות במסמך הכללי של הפרויקט,** עמדו עם עכבר ימני על הספריה **BlApi** **והוסיפו ממשק ציבורי,** ע"פ הכללים הבאים.

הנחיות כלליות להגדרת ממשק לוגי:

- כל ממשק לוגי יוגדר כ **interface**, **בקובץ נפרד** תחת תת-ספריה **BlApi** ולכן גם תחת מרחב שמות **BlApi**
- שם הממשק ושם הקובץ יהיה זהה לשם ישות השירות הלוגית בתוספת האות **I** גדולה בתחילתו.
- הרשאת הממשק תהיה **public**
- כל ממשק **יתועד לפניו** בעזרת ההערה המפורמטת ///. כמו בשלב 1.

```
namespace BlApi;
public interface IStudent
{
}
```

- **עבור כל אחת מישויות השירות הלוגיות (כולל ממשק הניהול IAdmin),** הגדירו את הפונקציונליות (המתודות) שהוא מאפשר מכיוון התצוגה.
  - הצעות למתודות עבור כל ישות שירות לוגית **מופיעות במסמך הכללי של הפרויקט**.
  - לא כל המתודות מופיעות במסמך הכללי של הפרויקט ותוכלו להוסיף מתודות על פי הצורך.

- **ממשק הניהול (IAdmin) שונה מעט משאר ממשקי השירות**, פירוט המתודות הנדרשות בו תוכלו למצוא במסמך הכללי של הפרויקט. באופן כללי הוא צריך להכיל את הפונקציונליות הבאה:
  - מתודה לאיפוס בסיס נתונים בשם **ResetDB**
  - מתודה לאתחול בסיס נתונים בשם **InitializeDB**
  - מתודת בקשה של שעון המערכת (הזמן הנוכחי של המערכת) בשם **GetClock**
  - מתודה לקידום שעון המערכת בשם **ForwardClock**
    - מקבלת כפרמטר את יחידת הזמן לקידום הזמן הנוכחי (דקה, שעה, יום, שנה)
    - ומקדמת את שעון המערכת ביחידת הזמן המתאימה
    - תוך שימוש בסופו של דבר במתודה AdminManager.UpdateClock().
  - מתודת בקשת משתני תצורה בשם **GetConfig**
    - לא מקבלת פרמטרים
    - מחזירה את ערכי משתני התצורה בתוך אובייקט מטיפוס BO.Config
      1. רק את משתני התצורה שרלוונטים כלפי מעלה לשכבת התצוגה (לא את המשתנים שהם לשימוש פנימי כגון מספרים רצים)
  - מתודת קביעת משתני תצורה בשם SetConfig
    - מקבלת אובייקט מטיפוס BO.Config שמכיל את כל ערכי משתני התצורה
      1. רק את אלו שמגיעים מכיוון התצוגה ולא את אלו שהם לשימוש פנימי כגון מספרים רצים
    - לא מחזירה ערך
    - מעדכנת את כל משתני התצורה
      1. רק אלו שהשתנו מאז העדכון הקודם

הנחיות כלליות להגדרת מתודות של ממשק לוגי:

- הוסיפו לממשק **הצהרות (הצהרות בלבד, ללא מימוש) של מתודות:**
  - **כשתגיעו למימוש הממשק יש לדאוג לכתיבה יעילה של קוד ולקוד קריא ומסודר**. כלומר, במידה ותרגישו צורך לייצר מתודת עזר פרטיות עבור קוד שחוזר על עצמו, יש להגדיר ולממש את המתודות אלו במחלקת העזר הלוגית שהגדרנו בשלב הקודם **ולא להגדירה כאן כמתודה של הממשק!**
- מתודות של הוספה או עדכון כולל של **ישות נתונים לוגית** - יקבלו כפרמטר מופע של **ישות נתונים לוגית** מתאימה
- מתודות של מחיקה או קבלה (אחזור) של **ישות נתונים לוגית** - יקבלו כפרמטר מספר מזהה של **ישות נתונים**
- מתודות פעולות ייחודיות (למשל עדכון סטטוס של ישות) יקבלו פרמטר(ים) לפי צורך ע"פ ראותכם
- כל הפרמטרים יועברו by-value (אסור להגדיר פרמטרים עם ref\out\in)
- המתודות יכולות להחזיר ערכים מהטיפוסים הבאים:
  - טיפוסים קיימים של דוט-נט (כגון int)
  - טיפוסים של ישויות BO - מופע של **ישות נתונים לוגית** מתאימה
  - אוספים של ישויות BO (רק ע"י IEnumerable גנרי) - **אוסף מופעים של ישות נתונים לוגית** מתאימה. אוסף כללי גנרי **IEnumerable** ולא List:
    - למשל `IEnumerable<BO.StudentInList> ReadAll( ...)`
- **המלצה חמה** - אל תוסיפו בראש הקובץ using namespace BO, על מנת שתאולצו לציין את מרחב השמות BO לפני כל שם של טיפוס ישות נתונים לוגית. כך יקל עליכם (בעיקר בהמשך) להבדיל בין ישות נתונים לוגית (BO) לישות נתונים (DO). למשל: להבדיל בין BO.Student לבין DO.Student.
  - לפעמים, תוך כדי העבודה מרחב השמות נוסף אוטומטית, וזה יכול לקרות כמה פעמים, המלצה חמה - תמיד להסיר אותו.
- המתודות לא חייבות להיות זהות בין ישות שירות לוגית אחת לשניה. אין ממשק מתודות מוסכם כמו שהיה בממשק ICrud בשכבת ה- DAL.

דוגמאות חלקיות:

```
namespace BlApi;
public interface IStudent
{
        void Create(BO.Student boStudent);
        BO.Student? Read(int id);

        IEnumerable<BO.StudentInList> ReadAll(BO.StudentFieldSort? sort = null, BO.StudentFieldFilter?
filter = null, object? value = null);
        void Update(BO.Student boStudent);
        void Delete(int id);

        void RegisterStudentToCourse(int studentId, int courseId);
        void UnRegisterStudentFromCourse(int studentId, int courseId);

        IEnumerable<BO.CourseInList> GetRegisteredCoursesForStudent(int studentId, BO.Year year =
BO.Year.None);
        IEnumerable<BO.CourseInList> GetUnRegisteredCoursesForStudent(int studentId, BO.Year year =
BO.Year.None);

        BO.StudentGradeSheet GetGradeSheetPerStudent(int studentId, BO.Year year = BO.Year.None);
        void UpdateGrade(int studentId, int courseId, double grade);
}

namespace BlApi;
public interface IAdmin
{
        void ResetDB();
        void InitializeDB();
        DateTime GetClock();
        void ForwardClock(BO.TimeUnit unit);
        BO.Config GetConfig();
        void SetConfig(BO.Config config);
}
```

בהמשך המסמך **נממש** את המתודות שהוגדרו בממשקים הללו.

8ב. הגדרת טיפוס ממשק לוגי ראשי בשם IBl

המטרה: הוספת ממשק לוגי ראשי בשם **IBl** שירכז את כל ממשקי השכבה הלוגית.

1. עמדו עם עכבר ימני על הספריה **BlApi** **והוסיפו בקובץ נפרד ממשק ציבורי** בשם IBl
2. הרשאת הממשק תהיה **public**
3. עבור כל אחד מהממשקים הלוגים (ישויות השירות הלוגיות) שהגדרנו בסעיף הקודם, הוסיפו תכונה מטיפוס **אותו ממשק**, **לקריאה בלבד** (רק עם get), **ללא אתחול** בהצהרה.

```
namespace BlApi;
public interface IBl
{
        IStudent Student { get; }
        ICourse Course { get; }
        IAdmin Admin { get; }
}
```

פרק 9 - מימוש ממשקי השכבה הלוגית (מימוש חוזי השירות הלוגיים)

לאחר שהגדרנו בפרק הקודם (פרק 8) את כל **ישויות השירות הלוגיות** (ממשקי השכבה הלוגית) והמתודות שלהן, ולאחר שהגדרנו וממשנו בפרק 7 את כל **מחלקות העזר לניהול לוגי** של ישויות נתונים. כעת, בפרק הנוכחי, נממש את מתודות הממשקים על פי כללים מסודרים, תוך שימוש במתודות העזר מפרק 7.

כאמור, השכבה הלוגית לא שומרת אצלה נתונים של ישויות BO ולכן לא צריכה להגן עליהם במעבר בין השכבות. כלומר, **המופעים המקומיים** של ישויות הנתונים הלוגיות שתיצרו בתוך המתודות שממשות את תתי-הממשקים בשכבה זו לא נשמרים בשום בסיס נתונים, אלא מחושבים מקומית בתוך המתודות ואז מועברים לטיפול לשכבה שמעל או לשכבה שמתחת. לכן, הישויות הלוגיות הוגדרו כ reference types מסוג **class**.

9א. הגדרת מחלקה מממשת עבור כל ממשק לוגי (ישות שירות לוגית)

נגדיר **עבור כל ממשק לוגי**, מחלקה **שתממש את המתודות שלו.** לצורך כך, בפרויקט BL, בצעו את הצעדים הבאים **עבור כל ממשק לוגי:**

1. תחת תת-ספריה **BlImplementation**, צרו מחלקה חדשה בהרשאת **internal**. כאשר שמה של המחלקה/הקובץ יהיה כשמו של הממשק הלוגי המתאים בצירוף המילה **Implementation**:
   a. למשל, אם שם ישות השירות הלוגי הוא Student אז שם המחלקה/קובץ יקרא StudentImplementation
2. הסירו מראש הקובץ את רשימת ה-using הגלובאליים הקיימת וודאו שמרחב השמות הוא **BlImplementation**.
3. הוסיפו בסוף שורת הכותרת של המחלקה ציון לכל **שהמחלקה יורשת מהממשק** המתאים לה:
   a. למשל, אם שם הישות הוא Student אז המחלקה הממשת תראה **בשלב זה** כך:

```
namespace BlImplementation;
using BlApi;
internal class StudentImplementation : IStudent
{
}
```

4. כעת, תוכלו סוף כל סוף, **לממש את כל הלוגיקה של המערכת שלנו** - כלומר לממש את המתודות על פי הכללים שהזכרנו בסעיף הקודם. **מתוך המימוש, על פי הצורך תתבצענה פניות למתודות של מחלקות העזר הסטטיות:**

   a. בכל פעם שתזהו מתוך המימוש קוד שחוזר על עצמו, או קוד מורכב שכדאי להפריד למתודת עזר - יש להגדיר ולממש את הקוד הזה כמתודת עזר בחלקת העזר המתאימה לאותה ישות נתונים (כפי שהוסבר בפרק 7).

   b. לאחר שהגדרתם ומימשתם את מתודות העזר במחלקת העזר, יש לזמן אותה במקום המתאים מתוך מתודות מימוש הממשק. למשל כך:

```
public void RegisterStudentToCourse(int studentId, int courseId)
        => StudentManager.LinkStudentToCourse(studentId, courseId);
```

5. **במימוש מחלקת הניהול** AdminImplementation:
   a. **ResetDB**: כדי לממש את **מתודות איפוס בסיס הנתונים** יש לקרוא למתודה ResetDB של AdminManager:

```
AdminManager.ResetDB();
```

b. **InitializeDB:** כדי לממש את **מתודות אתחול בסיס הנתונים** יש לקרוא למתודה InitializeDB של AdminManager:

```
InitializeManager.ResetDB();
```

c. **GetClock: גישה לשעון** המערכת לצורך **קבלת הזמן הנוכחי** תהיה אך ורק דרך מחלקת העזר AdminManager:

```
AdminManager.Now
```

d. **ForwardClock: גישה לשעון** המערכת לצורך **קידומו ביחידת זמן** כלשהי תהיה אך ורק דרך מחלקת העזר AdminManger. המתודה תקבל כפרמטר את יחידת הזמן לקידום. למשל אם קבלה בקשה לקדם בדקה:

```
AdminManager.UpdateClock(AdminManager.Now.AddMinutes(1));
```

e. **GetConfig:**

```
public BO.Config GetConfig() => AdminManager.GetConfig();
```

f. **SetConfig:**

```
public void SetConfig(BO.Config configuration) =>  AdminManager.SetConfig(configuration);
```

דוגמאות למימוש מתודות של ממשקים לוגיים, למערכת הסטודנטים-קורסים שבעזרתה אנחנו מדגימים לכל אורך השלבים, ניתן לראות כאן: נספח 3 - דוגמאות למימוש מתודות של ממשק לוגי ע"פ הכללים

9ב. הגדרת מחלקה חדשה בשם Bl שתממש את IBl

לאחר שממשנו את כל הממשקים הלוגים וגם יצרנו ממשק **IBl** שמרכז את כל תתי-הממשקים הללו. נרצה כעת, ליצור מחלקה ראשית בשם **Bl** עבור השכבה הלוגית שתממש את הממשק הלוגי הראשי **IBl**.

לצורך כך, בפרוייקט BL:

1. תחת תת-ספריה **BlImplementation**, צרו מחלקה בשם **Bl**, עם הרשאת **internal**, שתירש מהממשק הלוגי הראשי **IBl**.
   a. שימו לב - ההרשאה היא **internal** ולא **public** מכיוון, שבסעיף הבא נהפוך את השכבה BL כולה **לפקטורי** והיא לא תצטרך להיות חשופה כלפי השכבה שמעליה.

```
namespace BlImplementation;
using BlApi;
internal class Bl : IBl
```

2. **עבור כל אחד מהממשקים הלוגים (כולל ממשק הניהול IAdmin)** ממשו בתוך המחלקה **Bl** את התכונה (property) המתאימה שהוגדרה עבורו בממשק **IBl**, כך שתאותחל בזמן הגדרתה (עם get בלבד) באובייקט מהטיפוס שממשמש את הממשק המתאים לאותה ישות לוגית.
   a. לדוגמא, עבור תת ממשק IStudent, נוסיף למחלקה תכונה כך:

```
public IStudent Student { get; } = new StudentImplementation();
```

פרק 10 - הגדרת מחלקת יצרן (Simple Factory) עבור השכבה הלוגית

כזכור, בתחילת מסמך זה הגדרנו מחלקת **Factory** עבור שכבת הנתונים DAL. כעת, נרצה להגדיר **מחלקה דומה עבור השכבה הלוגית BL.**

נרצה ששכבת התצוגה, אשר היא מעל השכבה הלוגית, תוכל **לייצר אובייקטים מסוג Bl מבלי להכיר את המחלקה המממשת BlImplementation** אלא רק את **הממשק המוסכם IBl**.

המחלקת Factory שהגדרנו בשכבת נתונים DAL, קראה את שם המחלקה המממשת שעליה לאתחל **מקובץ קונפיגורציה.**

לעומת זאת, בשכבה הלוגית BL, ניצור מחלקת Factory פשוטה יותר שמאתחלת את המחלקה המממשת **במפורש דרך הקוד**. כלומר, השכבה שמעל לא תכיר את המחלקה המממשת, אך השכבה שמתחת תצטרך להתקמפל מחדש במידה והמחלקה המממשת תשתנה.

לצורך כך, בפרוייקט BL, הגדירו מחלקת יצרן - "מפעל לייצור אובייקטים":

1. תחת תת-ספריה **BlApi**, צרו מחלקה **ציבורית סטטית** בשם **Factory**

```
namespace BlApi;
public static class Factory
```

2. הגדירו בתוך המחלקה מתודה ציבורית סטטית בשם **Get** של המפעל שתחזיר עצם מאותחל מטיפוס הממומש את הממשק **IBl**.

```
public static IBl Get() => new BlImplementation.Bl();
```

פרק 11 - הגדרת חריגות עבור השכבה הלוגית בתוך פרויקט BL

מהלך הטיפול בחריגות בשכבת הנתונים BL:

- שכבת BL **תתפוס** חריגות **שיזרקו כלפיה מלמטה משכבת הDAL** שמתחתיה. כלומר, שכבת ה DAL תזרוק את אותן טיפוסי חריגות שכבר הגדרנו ב **DO** בפרויקט DalFacade. **כל חריגה כזו שנתפסה בשכבת BL:**
  - **תייצר חריגה חלופית** מטיפוס חדש שיוגדר ב **BO**
  - **החריגה החלופית** תכלול בתוכה את החריגה שהתקבלה משכבת הנתונים **כחריגה פנימית**
  - **ותזרוק** אותה **כלפי שכבת התצוגה PL** שמעליה
- בנוסף, עבור מצבים של **תקלה פנימית** בתוך השכבה הלוגית:
  - שכבת BL **תייצר חריגה מתאימה** מטיפוס חדש שיוגדר ב **BO**
  - **ותזרוק** אותה **כלפי שכבת התצוגה PL** שמעליה

11א. הגדרת חריגות עבור שכבת BL

נגדיר כעת תחת ספריית BO, שני סוגים של חריגות עבור שכבת BL:

- עבור כל טיפוס חריגת DO, נגדיר ב BO טיפוס חריגה **חלופית ש"מקבילה"** לחריגה ב DO
  - גם כאן, כמו ב DO, החריגות לא יהיו ספציפיות לסוג של ישות לוגית אלא לסוג של תקלה
    - למשל: מספר מזהה כבר קיים, לא ניתן למחוק ישות זו וכדומה
- עבור מצבים **של תקלה פנימית** בתוך השכבה הלוגית, נגדיר טיפוס חריגה עבור אותה **תקלה פנימית**
  - למשל: ערך נתון לא תקין, סדר תאריכים לא נכון וכדומה

לצורך הגדרת טיפוסי חריגות בשכבת הנתונים:

1. תחת ספריית **BO** בפרויקט BL, צרו קובץ בשם **Exceptions.cs**
2. הסירו מראש הקובץ את רשימת ה-using הקיימת וודאו שמרחב השמות הוא **BO** עם נקודה פסיק **;** .
3. **רוקנו** את הקובץ מהגדרת המחלקה שנוצרה, כיוון שאתם הולכים להגדיר בתוכו מחלקות חדשות אחרות
4. **עבור כל טיפוס חדש של חריגה, הגדירו מחלקה נפרדת, כאשר שם המחלקה יתחיל במילה Bl**
5. כל מחלקות החריגה **יופיעו בקובץ הזה אחת אחרי השניה**

דוגמה להגדרה של טיפוס חדש של חריגה מסוג "האובייקט לא קיים":

החריגה תיזרק למשל כאשר BL מנסה, דרך פניה ל DAL, לעדכן אובייקט עם מספר מזהה **שלא קיים** ברשימת האובייקטים מסוג כלשהו. ואשר בעקבותיו, DAL זרקה חריגה משלה. בתגובה BL, זורק חריגה מקבילה כלפי מעלה ל PL.

```
namespace DO;
[Serializable]
public class BlDoesNotExistException : Exception
{
        public BlDoesNotExistException(string? message) : base(message) { }
        public BlDoesNotExistException(string message, Exception innerException)
                : base(message, innerException) { }
}
```

דוגמה להגדרה של טיפוס חדש של חריגה מסוג "תכונה עם ערך null":

החריגה תיזרק למשל כאשר מנסים ב BL להשתמש בערך של תכונה שערכו הוא null.

```
[Serializable]
public class BlNullPropertyException : Exception
{
        public BlNullPropertyException(string? message) : base(message) { }
}
```

דוגמא לתפיסה של חריגת DO וזריקה של חריגת BO "מקבילה":

ניתן לראות כאן: נספח 3 - דוגמאות למימוש מתודות של ממשק לוגי ע"פ הכללים

פרק 12 - בניית תוכנית בדיקה BlTest

כעת, בתוכנית הראשית (Main) של פרויקט BlTest שיצרתם בתחילת המסמך, נוסיף קוד לבדיקת הפונקציונאליות של השכבה הלוגית, ע"פ הכללים הבאים:

- לתוכנית **אסור** לכלול שום לוגיקה אלא רק:
  - קלט
  - בקשות משכבה לוגית (קריאה למתודות של ממשקי IBl)
  - פלט
- **מותר** לבצע בדיקות תקינות פורמט של קלט (כגון כשצריך מספר אבל הקלידו תווים שאינן ספרות)
- את בדיקת הלוגיקה תבצע שכבת ה BL שזה עתה כתבתם ולא תוכנית הבדיקה BlTest

1. הוסיפו למחלקה הראשית **Program** שדה בשם s_bl מסוג הממשק של השכבה הלוגית IBl.
2. אתחלו את השדה בעזרת מחלקת היצרן Factory שתאתחל את מחלקת המימוש המתאימה בפרויקט BL.

```
static readonly BlApi.IBl s_bl = BlApi.Factory.Get();
```

3. דרך שדה זה, תפנו לממשקים הלוגיים ב BL, לצורך בדיקת הפונקציונאליות של השכבה הלוגית:

```
BO.Student?  boStude = s_bl.Student.Read(3);
```

4. **מבנה התוכנית הראשית דומה לתוכנית הבדיקה של שכבת DAL, רק שהיא תבדוק את הממשק הלוגי:**
   - בתוכנית הראשית Main, השתמשו בלולאה שתדפיס בקונסול את התפריט הראשי למשתמש לטיפול בכל אחד מישויות השירות הלוגיות (**הממשקים הלוגיים**, **כולל ממשק הניהול IAdmin**) ובנוסף גם אפשרות יציאה מהתוכנית.
   - עבור כל ישות שירות לוגית, הגדירו במחלקה הראשית מתודה פרטית נפרדת שתציג תת-תפריט עבור כל הפעולות שהוגדרו בממשק שלה ובנוסף גם אפשרות יציאה מתת-התפריט בחזרה לתפריט הראשי. חובה לאפשר בתכנית להפעיל את כל הפעולות של ממשקי השכבה הלוגית.
   - עבור כל אפשרות בתת-תפריט של הישות:
     - קלטו את כל הנתונים הרלוונטיים הדרושים לזימון הפעולה
     - הכינו את הנתונים ואת מופעי ה BO הנדרשים לפעולה
     - זמנו את הפעולה דרך מתודת הממשק המתאים (דרך השדה s_bl)
     - הדפיסו את תוצאות המתודה, בהתאם לסוג הערך המוחזר
5. **הדפסות**:
   - אובייקטים של ישויות נתונים יודפסו בעזרת מתודת Console.WriteLine שתזמן באופן עקיף את ToString. אסור לזמן את המתודה ToString בצורה מפורשת!
   - אוסף יודפס בעזרת לולאת foreach בלבד ובתוכה הדפסת אובייקטים כנ"ל
6. **חריגות**:
   - תפסו את החריגות הצפויות מסוג BO (עבור הפעולה המסוימת) של שכבת BL.
   - עבור כל חריגה שנתפסה נדפיס את המידע על החריגה: שם החריגה, הודעה שבחריגה, מידע על חריגה פנימית (אם יש)
7. **תוספת**: לצורך המרות של מספרים ותאריכים מהקלט - להשתמש במתודה TryParse. תוספת זו מוכרת רק אם נעשה בה שימוש מלא, כלומר:
   a. נבדק ערך מוחזר (לבדיקת תקינות הקלט)
   b. בכל מקום שאפשר המשתנה עבור קבלת ערך מוגדר בתוך הזימון ל TryParse

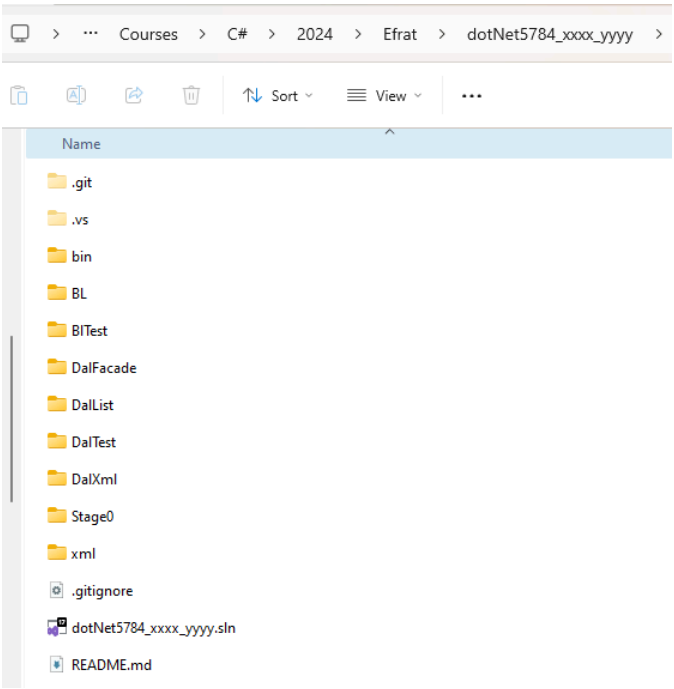
תוכלו לראות דוגמאות שימושיות בעמוד הבא.

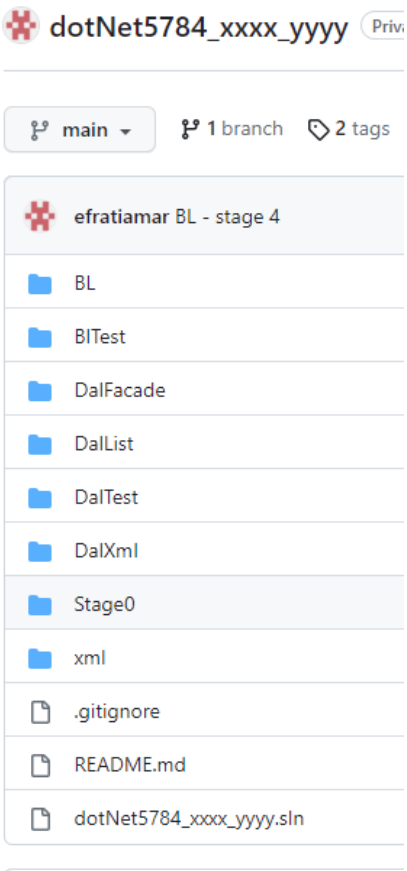
אוסף דוגמאות לקריאות של מתודות דרך ממשקים לוגיים, כחלק מהתפריט הראשי ומתתי התפריטים:

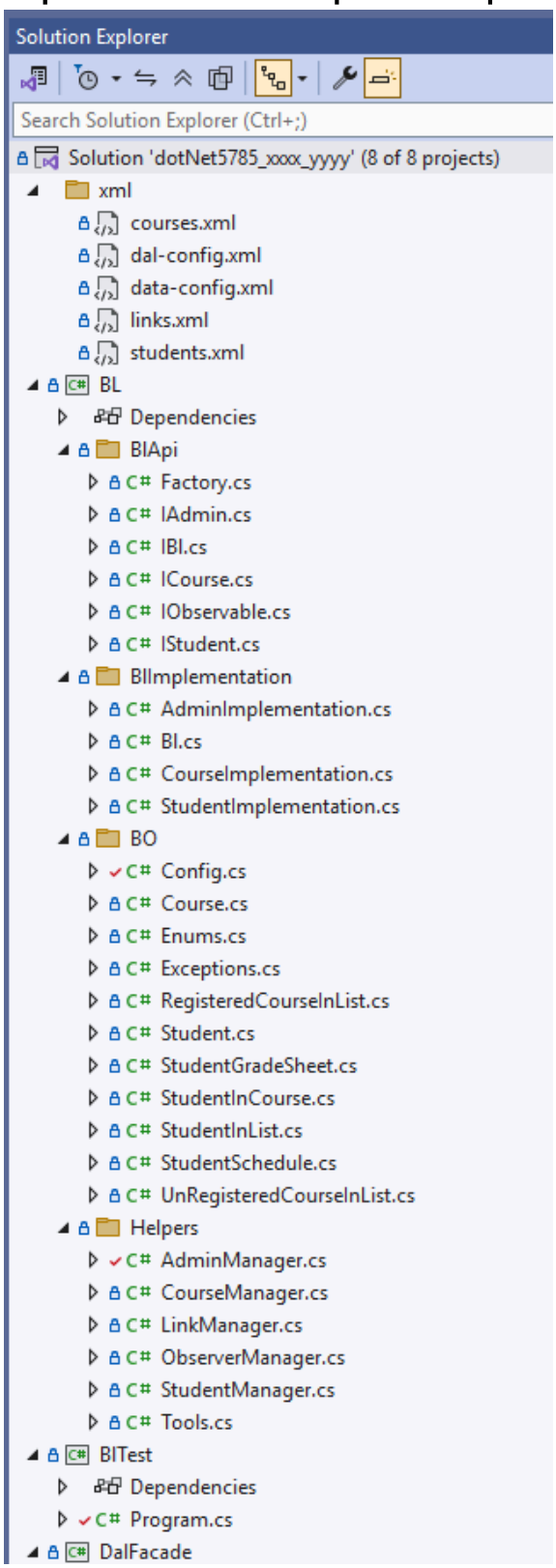
```
s_bl.Admin.ResetDB();

s_bl.Admin.InitializeDB();

s_bl.Admin.ForwardClock(BO.TimeUnit.HOUR);

Console.WriteLine(s_bl.Admin.GetClock());

var configuration = s_bl.Admin.GetConfig();
Console.WriteLine("Configuration:");
Console.WriteLine($"MAX_RANGE = {configuration.MaxRange}");
//...

int newMaxRange = 5;
s_bl.Admin.SetConfig(new BO.Config() { MaxRange = newMaxRange });
configuration = s_bl.Admin.GetConfig();
Console.WriteLine("Configuration:");
Console.WriteLine($"MAX_RANGE = {configuration.MaxRange}");
//...


try
{
        BO.Student? boStud = s_bl.Student.Read(239894668);
        Console.WriteLine(boStud);
}
catch (BO.BlDoesNotExistException ex)
{
        Console.WriteLine(ex);
}

//...
foreach (var item in s_bl.Student.ReadAll())
        Console.WriteLine(item);
```

פרק 13 - תמונת מצב

לאחר ביצוע כל הפרקים של שלב 4, המאגר ב-github.com, ה-Solution ב-Visual Studio, והתיקיות בסייר הקבצים אמורים להיראות כך:







פרק 14 - יצירת תג והגשת שלב 4 במודל

ע"פ ההנחיות שמוסברות בשלב 0:

- צרו תג בשם: **Stage4 Final commit**
- הגישו קישור לתג במודל של **כל אחד מבני הזוג**

סוף שלב 4 - בהצלחה 🙂

נספחים - שלב 4

נספח 1 - דוגמאות לישויות נתונים לוגיות

דוגמא לישות נתונים לוגית עבור סטודנט BO.Student המבוססת על ישות נתונים DO.Student משכבת נתונים:

נרצה להציג אותה למשל בשכבת התצוגה במסך שמאפשר פעולות (עדכון/מחיקה/יצירה) של סטודנט בודד.

שימו לב לתכונה **CurretnYear** שלא היתה קיימת בישות **DO.Student**, תכונה התלויה בזמן הרישום, אשר ערכה משתנה משנה לשנה. תוצג רק בתצוגה ותשמש גם לכל מיני חישובים שקשורים בתצוגה בהמשך. למשל האם ניתן לרשום את הסטודנט לקורס מסוים על סמך השנה שבה נמצא הסטודנט וכדומה.

```
using Helpers;
namespace BO;
public class Student
{
        public int Id { get; init; }
        public string? Name { get; set; }
        public string? Alias { get; set; }
        public bool IsActive { get; set; }
        DateTime? BirthDate { get; set; }
        public DateTime RegistrationDate { get; init; }
        Year CurrentYear { get; init; } //(BO.Year)(AdminManager.Now.Year - registrationDate?.Year!);
        public override string ToString() => this.ToStringProperty();
}
```

דוגמא לישות נתונים לוגית BO.StudentInCourse שמרחיבה את הישות DO.Link עם תכונות נוספות שקשורות לאותו קורס שהסטודנט רשום אליו:

נרצה להציג אותה למשל בשכבת התצוגה במסך המאפשר הצגת/עדכון ציון לסטודנט שרשום לקורס מסוים. או כחלק מגיליון ציונים של סטודנט.

```
using Helpers;
namespace BO;
public class StudentInCourse
{
        public int StudentId { get; init; }
        public required Tuple<int, string, string> Course { get; init;
        public Year? InYear { get; init; }
        public SemesterNames? InSemester { get; init; }
        public double? Grade { get; set; }
        public int? Credits { get; init; }
        public override string ToString() => this.ToStringProperty();
}
```

דוגמא לישות נתונים לוגית עבור גיליון ציונים של סטודנט BO.StudentGradeShhet המבוססת על הרבה חישובים ושאילתות שנאספו מבסיס הנתונים:

נרצה להציג אותה למשל בשכבת התצוגה במסך המאפשר הצגת גיליון ציונים עבור סטודנט מסוים.

שימו לב לתכונה StudentAverage שהיא תוצאה של שאילתה מורכבת שמחשבת את הממוצע עבור כל הקורסים שהסטודנט רשום אליהם.

שימו לב גם לתכונה Courses שהיא תוצאה של שאילתה המחזירה את רשימת כל הקורסים שהסטודנט רשום אליהם. קורס ברשימה הוא מטיפוס ישות נתונים לוגית אחרת BO.StudentInCourse.

```
using Helpers;
namespace BO;
public class StudentGradeSheet
{
        public int StudentId { get; init; }
        public string? StudentName { get; init; }
        public double? StudentAverage { get; init; }
        public List<StudentInCourse>? Courses { get; init; } = null;
        public override string ToString() => this.ToStringProperty();
}
```

דוגמא לישות נתונים לוגית BO.CourseInList המכילה פרטים מצומצמים על קורס, היא אמורה להופיע כחלק מרשימה של קורסים:

נרצה להציג אותה למשל בשכבת התצוגה במסך המאפשר הצגת רשימת כל הקורסים הקיימים באוניברסיטה כולה, עם כמה שפחות פרטים. כאשר לחיצה על כל קורס מובילה לתצוגה של פרטים מלאים על הקורס.

```
namespace BO;
public class CourseInList
{
        public int Id { get; init; }
        public required string CourseNumber { get; set; }
        public required string CourseName { get; set; }
        public Year? InYear { get; init; }
        public SemesterNames? InSemester { get; init; }
        public override string ToString() => this.ToStringProperty();
}
```

נספח 2 - דוגמאות להגדרה ומימוש של מתודות עזר לוגיות

דוגמא למתודות עזר במחלקת StudentManager שנוצרה עבור שימוש בקוד שחוזר על עצמו (יותר יעיל להוציאו למתודה נפרדת):

```
private static readonly IDal s_dal = Factory.Get;

internal static BO.Year GetStudentCurrentYear(DateTime? registrationDate)
{
        BO.Year currYear = (BO.Year)(AdminManager.Now.Year - registrationDate?.Year!);
        return currYear > BO.Year.None ? BO.Year.None : currYear;
}
```

דוגמא למתודות עזר במחלקת StudentManager שנוצרה עבור קוד בעל לוגיקה מורכבת (יותר קריא להוציאו למתודה נפרדת):

```
internal static BO.Student? GetStudent(int id)
{
        DO.Student doStudent;
        doStudent = s_dal.Student.Read(id) ?? throw new BO.BlDoesNotExistException($"Student with
ID={id} does Not exist");

        return new BO.Student()
        {
        Id = id,
        Name = doStudent.Name,
        Alias = doStudent.Alias,
        IsActive = doStudent.IsActive,
        BirthDate = doStudent.BirthDate,
        RegistrationDate = doStudent.RegistrationDate,
        CurrentYear = GetStudentCurrentYear(doStudent.RegistrationDate),
        Email = doStudent.Email,
        MaxDistance = doStudent.MaxDistance,
        Address = doStudent.Address,
        Latitude = doStudent.Latitude,
        Longitude = doStudent.Longitude
        };
}
```

```
internal static BO.StudentInCourse GetDetailedCourseForStudent(int studentId, int courseId)
{
        DO.Link? doLink = s_dal.Link.Read(l => l.StudentId == studentId && l.CourseId == courseId)
        ?? throw new BO.BlDoesNotExistException($"Student with ID={studentId} does Not take Course
with ID={courseId}");
        DO.Course? doCourse = s_dal.Course.Read(courseId)
         ?? throw new BO.BlDoesNotExistException($"Course with ID={courseId} does Not exist");

        return new BO.StudentInCourse()
        {
        StudentId = studentId,
        Course = new Tuple<int, string, string>(doCourse.Id, doCourse.CourseNumber,
doCourse.CourseName),
        InYear = (BO.Year?)doCourse.InYear,
        InSemester = (BO.SemesterNames?)doCourse.InSemester,
        Grade = doLink.Grade,
        Credits = doCourse.Credits
        };
}
```

דוגמא למתודות עזר במחלקת StudentManager שנוצרה עבור קוד פנימי שאינו חלק מהממשק הציבורי:

מתודה לצורך עדכון מצב תקופתי - מתודה שמופעלת לאחר כל עדכון של שעון המערכת ובודקת האם יש סטודנטים שמספר השנים שהם לומדים עובר את ערכו של משתנה התצורה MaxRange, וכתוצאה מכך מעדכן את הסטודנט להיות 'לא פעיל'.

```
internal static void PeriodicStudentsUpdates(DateTime oldClock, DateTime newClock)
{
        var list = s_dal.Student.ReadAll().ToList();
        foreach (var doStudent in list)
        {
                //if student study for more than MaxRange years
                //then student should be automatically updated to 'not active'
                if (AdminManager.Now.Year - doStudent.RegistrationDate?.Year >= s_dal.Config.MaxRange)
                {
                        s_dal.Student.Update(doStudent with { IsActive = false });
                }
        }
}
```

נספח 3 - דוגמאות למימוש מתודות של ממשק לוגי ע"פ הכללים

```
using BlApi;
using Helpers;
namespace BlImplementation;
internal class StudentImplementation : IStudent
{
        public void Create(BO.Student boStudent)
        {
                StudentManager.CreateStudent(boStudent);
        }

        public BO.Student? Read(int id)
        {
                return StudentManager.GettStudent(id);
        }
        //...
        public void RegisterStudentToCourse(int studentId, int courseId)
        {
                StudentManager.LinkStudentToCourse(studentId, courseId);
        }

        //...
```

```
        public void UpdateGrade(int studentId, int courseId, double grade)
        {
                StudentManager.UpdateCourseGradeForStudent(studentId, courseId, grade);
        }
        //...
        public BO.StudentGradeSheet GetGradeSheetPerStudent(int studentId, BO.Year year=BO.Year.None)
        {
                return StudentManager.GetGradeSheetPerStudent(studentId, year);
        }
}
```